Stored XSS
Code Walkthrough

Cyber Security · IT-30001 · Autumn Semester 2026

Purpose — This guide explains the code path behind the local Stored Cross-Site Scripting demonstration, the root cause, and the implemented mitigation. It is designed to be used alongside the source files during your viva.

01 · Understand the flow

STEP	WHAT HAPPENS
1 · Submit	The browser sends the comment to POST /posts.
2 · Store	Node.js appends it to data/posts.json.
3 · Retrieve	A later GET / request reads the saved comments.
4 · Render	The selected mode determines whether the message is encoded before insertion into HTML.
5 · Browser	Raw markup can be interpreted in vulnerable mode; encoded characters display as text in protected mode.

02 · The vulnerable renderer

In vulnerable mode, the stored comment is inserted into the HTML template without encoding. The browser therefore parses the supplied text as document markup.

```
const message = post.message;

return `<div class="post-message">
  ${message}
</div>`;
```

Explain: The key problem is not that the application accepts a comment. The problem is that it later treats untrusted comment data as HTML structure.

03 · The protected renderer

In protected mode, the message is encoded before it is inserted into the HTML text node. The original comment remains stored; only its rendered representation changes.

```
const message = escapeHtml(post.message);

return `<div class="post-message">
  ${message}
</div>`;
```

04 · The encoding helper

```
function escapeHtml(value) {
  return String(value)
    .replaceAll('&', '&amp;')
    .replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;')
    .replaceAll('\"', '&quot;')
    .replaceAll("'", '&#39;');
```

```
}
```

The ampersand is encoded first. This prevents the ampersands introduced by the other entities from being encoded again.

05 · How to present the demonstration

• Open the local app and select Vulnerable mode.

• Click Insert safe demo payload, then publish the comment.

• Show the alert and point out that the post is now persisted.

• Switch to Protected mode and show the exact same value as literal text.

• Open server.js and compare the two branches in renderPosts().

• Finish by explaining why output encoding is context-specific.

06 · Key viva answers

Why Stored XSS? The payload is saved by the application and executes when a later page view renders that stored value.

Why not rely on input validation? Length or format validation does not ensure data is safe in every output context. Correct output handling is still required.

Is this helper universal? No. It is for plain text in an HTML element. JavaScript, CSS, URL and attribute contexts require context-specific handling.

What does HttpOnly do? It prevents JavaScript from reading a cookie via document.cookie. It does not prevent XSS execution.

What does CSP do? It restricts script execution and sources as defence-in-depth; it does not replace safe output encoding.

Safety note: This is a localhost-only educational lab. Use only the harmless alert payload. Do not deploy the intentionally vulnerable version or attempt to access real credentials or cookies.